Vidyanidhi PU College Website Project

Development-to-Deployment Guide with AWS Cost Estimate
Node.js + Express | React + Tailwind CSS | MongoDB | SMTP | AWS

Prepared: 09 October 2026

Scope of phase 1: Public website (Home, About Us, Courses, Facilities, Blogs, News, Results, V-Mastermind, Contact), an admission enquiry form that stores student name, course, phone number and address in MongoDB, and emails every enquiry to the college mailbox through SMTP.

Included with this document: the starter source code (`vidyanidhi/backend` and `vidyanidhi/frontend`) built to this guide.

Cost figures marked **(sourced)** come from a published price lookup; the rest are planning estimates. AWS and MongoDB Atlas prices change, so confirm in the AWS Pricing Calculator before committing.

1. Project overview and architecture

The site is a three-tier application. The browser loads a static React build from a CDN; the React app calls a versioned REST API; the API validates the request, writes it to MongoDB and sends a notification email to the college.

Layer	Technology	Role
Frontend	React 18, Vite, React Router, Tailwind CSS	Responsive UI, shared components (Header, Button, FormField, Container)
Backend	Node.js 20, Express 4	REST API under /api/v1, validation, rate limiting, security headers
Database	MongoDB (Atlas) with Mongoose	Stores enquiries; visible in Atlas / Compass
Email	Nodemailer over SMTP	Sends each enquiry to the college mailbox
Hosting	AWS: S3 + CloudFront (frontend), EC2 + Nginx + PM2 (backend), Route 53, ACM	Production deployment in Mumbai (ap-south-1)

Request flow for the enquiry form

```
Browser (React form)
   |  POST /api/v1/enquiries   {studentName, course, phoneNumber, address}
   v
Route -> Validator -> Controller -> Service -> Repository -> MongoDB
                                       |
                                       +--> Mail service -> SMTP -> college mailbox
```

The service saves the enquiry **first** and then sends the email, so a temporary SMTP outage never loses a student's details. A `mailSent` flag on each record shows which emails still need a retry.

2. Folder structure

Backend (separation of routes, controllers, services, repositories, models)

```
backend/
  package.json  .env.example  Dockerfile
  src/
    server.js                  start-up, DB connect, graceful shutdown
    app.js                     express app, helmet, cors, rate limit
    config/         env.js  db.js  mailer.js
    routes/         index.js  enquiry.routes.js
    controllers/    enquiry.controller.js
    validators/     enquiry.validator.js
    services/       enquiry.service.js  mail.service.js
    repositories/   enquiry.repository.js
    models/         enquiry.model.js
    middlewares/    validate.middleware.js  error.middleware.js
    utils/          ApiError.js  asyncHandler.js
```

Rule of thumb: **routes** map URLs, **controllers** handle HTTP only, **services** hold business rules, **repositories** are the only code that touches the database, and **models** define the schema. To add a feature (for example Blogs) you add one file to each folder and mount one line in `routes/index.js`.

Frontend (common components for styling)

```
frontend/
  index.html  tailwind.config.js  vite.config.js  .env.example
  src/
    main.jsx  App.jsx  index.css
    config/siteConfig.js        phones, email, nav links, course list
    api/        axiosClient.js  enquiry.api.js
    components/
      common/   Button  FormField  Container  SectionTitle
      layout/   TopBar  Header  Marquee  Footer  Layout
      EnquiryForm.jsx
    pages/      Home  About  Courses  Contact  Placeholder
```

The header reproduces the reference site: grey contact strip, purple navigation bar (Home, About Us, Courses, Facilities, Blogs, News Articles, Results, V-Mastermind, Contact, Admission Open button with dropdowns) and the gold scrolling admission banner. Brand colours live in `tailwind.config.js`, so a re-theme is a one-file change.

3. Development phases

Phase 1 - Requirements and planning (about 2 days)

- Confirm the page list, the exact course names and the college mailbox that will receive enquiries.
- Collect content and assets: logo, campus photos, faculty details, result data, brochures.
- Decide who will manage the data (admin view of enquiries) and who owns the AWS and MongoDB accounts.
- Agree the acceptance checklist in section 8.

Phase 2 - Environment setup (about 1 day)

```
# one-time tools: Node.js 20 LTS, Git, VS Code, MongoDB Compass
cd vidyanidhi/backend  && npm install && cp .env.example .env
cd ../frontend         && npm install && cp .env.example .env
```

- Create a free MongoDB Atlas cluster, add a database user, allow your IP, paste the connection string in `MONGO_URI`.
- Create a Git repository with `main` (production) and `develop` branches.

Phase 3 - Backend and enquiry API (about 4 days)

Method	Endpoint	Purpose	Notes
POST	/api/v1/enquiries	Submit the enquiry form	Public; validated; rate limited (20 / 15 min / IP)
GET	/api/v1/enquiries	List enquiries (paged)	Add admin login (JWT) before production
GET	/api/v1/enquiries/:id	One enquiry	Admin only in production
GET	/api/v1/health	Health check	Used by AWS and uptime monitors

Stored fields: studentName, course, phoneNumber (10-digit Indian mobile), address, mailSent, status (new / contacted / admitted / closed), createdAt, updatedAt.

Test the API locally:

```
curl -X POST http://localhost:5000/api/v1/enquiries \
  -H "Content-Type: application/json" \
  -d '{"studentName":"Test Student","course":"1 PUC - Commerce",
       "phoneNumber":"9876543210","address":"Hubballi, Karnataka"}'
```

Viewing the data: open MongoDB Compass or Atlas, database `vidyanidhi`, collection `enquiries`. In phase 2 of the project (below) a small admin dashboard can show the same list inside the website.

Phase 4 - Frontend (about 8 days)

- Build layout shell and header first, then the Home hero with the enquiry form, then About Us, Courses, Facilities, Results, V-Mastermind, Blogs, News.
- Always build new UI from the common components so spacing, colours and form styles stay identical across pages.
- Check every page at mobile (360 px), tablet (768 px) and desktop (1280 px+).
- Set `VITE_API_URL` to the deployed API address before the production build.

Phase 5 - SMTP email setup (about 1 day)

Two practical options for the college mailbox:

Option	Setup	Limit / cost
Gmail / Google Workspace SMTP	Turn on 2-step verification, create an **App Password**, use smtp.gmail.com, port 465, secure=true	About 500 emails/day on free Gmail; free
Amazon SES (recommended for scale)	Verify the domain, request production access, create SMTP credentials, use the SES SMTP endpoint for ap-south-1	About $0.10 per 1,000 emails

Only the `.env` values change between the two; the code stays the same. Never commit `.env` to Git.

Phase 6 - Testing (about 3 days)

- API: valid request, missing fields, bad phone number, wrong course, oversized body, rate-limit trigger.
- Email: arrives in the college inbox (check spam), correct details, special characters in the name do not break the HTML.
- Database: record appears, `mailSent` becomes true; stop SMTP and confirm the record is still saved.
- Browser: Chrome, Edge, Safari, Android and iPhone; form success and error messages.

4. Deployment on AWS (Mumbai, ap-south-1)

Target architecture: Route 53 (DNS) -> CloudFront (CDN + HTTPS) -> S3 for the React build; api.vidyanidhipucollege.com -> EC2 (Nginx reverse proxy + PM2 running Node) -> MongoDB Atlas. SSL certificates come from AWS Certificate Manager and Let's Encrypt at no charge.

Step 1 - Secure the AWS account

- Enable MFA on the root user, create an IAM admin user, and stop using root.
- Set a billing alert (for example at Rs 3,000/month) in AWS Budgets.

Step 2 - Database (MongoDB Atlas)

- Create the cluster in the AWS Mumbai region. Start on M0 (free) for testing; move to M10 before launch for backups and reliability.
- Network access: allow only the EC2 server's Elastic IP. Create a dedicated app user with read/write on the `vidyanidhi` database only.

Step 3 - Backend on EC2

```
# Launch Ubuntu 22.04/24.04, t3.small, 20 GB gp3, attach an Elastic IP
# Security group: 22 (your IP only), 80, 443 (anywhere)

sudo apt update && sudo apt install -y nginx git
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - && sudo apt install -y nodejs
sudo npm i -g pm2

git clone <your-repo> && cd vidyanidhi/backend
npm ci --omit=dev
nano .env                       # production values, NODE_ENV=production
pm2 start src/server.js --name vidyanidhi-api
pm2 save && pm2 startup         # restart on reboot
```

Nginx reverse proxy (`/etc/nginx/sites-available/api`):

```
server {
  server_name api.vidyanidhipucollege.com;
  location / {
    proxy_pass http://127.0.0.1:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
# then:  sudo certbot --nginx -d api.vidyanidhipucollege.com
```

Step 4 - Frontend on S3 + CloudFront

```
cd frontend && npm run build                # creates dist/
aws s3 sync dist/ s3://vidyanidhipucollege-web --delete
aws cloudfront create-invalidation --distribution-id <ID> --paths "/*"
```

- S3 bucket stays private; CloudFront uses Origin Access Control to read it.
- Add a CloudFront custom error response: 403/404 -> `/index.html` with status 200 (needed for React Router).
- Request the certificate in **us-east-1** (required for CloudFront) and attach it with the domain vidyanidhipucollege.com and www.

Step 5 - DNS and CORS

- Route 53: alias A records for the apex and www to CloudFront; A record `api` to the Elastic IP.
- Backend `CORS_ORIGINS=https://vidyanidhipucollege.com,https://www.vidyanidhipucollege.com`; frontend `VITE_API_URL=https://api.vidyanidhipucollege.com/api/v1`.

Step 6 - Automated deployment (CI/CD)

Use GitHub Actions: on push to `main`, one job builds the React app and syncs it to S3 / invalidates CloudFront; another SSHs into EC2 (or uses AWS SSM) to run `git pull && npm ci --omit=dev && pm2 reload vidyanidhi-api`. Store AWS keys and the SSH key in GitHub Secrets, or better, use an OIDC role.

Step 7 - Security, monitoring and backups

- Already in the code: Helmet headers, CORS allow-list, request size limit, input validation, rate limiting on /api.
- Add: admin authentication (JWT) for the enquiry list, Google reCAPTCHA v3 on the form, and HTTPS only.
- Monitoring: CloudWatch agent for CPU / disk, an uptime check on `/api/v1/health`, PM2 log rotation.
- Backups: Atlas M10 continuous backup; weekly EBS snapshot of the server.
- Privacy: students' phone numbers are personal data. Add a privacy notice on the form and limit who can read the collection.

5. Cost estimate

Monthly figures are in US dollars with an assumed planning rate of **Rs 88 per USD** (change this if the rate differs). Region: AWS Mumbai (ap-south-1). Only the EC2 price is sourced; other lines are estimates for low traffic (a college website).

5.1 Monthly running cost

Item	USD / month	INR / month
EC2 t3.small (API server) (sourced: $0.0224/hr x 730 h)	16.35	Rs 1,439
EBS 20 GB gp3 disk	1.85	Rs 163
Public IPv4 address (Elastic IP)	3.65	Rs 321
S3 + CloudFront (low traffic)	2.00	Rs 176
Route 53 hosted zone + queries	0.60	Rs 53
CloudWatch basic monitoring / logs	2.00	Rs 176
Atlas M0 free cluster + Gmail SMTP + ACM SSL	0.00	Rs 0
Option A - Starter total	**26.45**	**Rs 2,328**

Item	USD / month	INR / month
Everything in Option A	26.45	Rs 2,328
MongoDB Atlas M10 (backups, dedicated) - estimate	57.00	Rs 5,016
Amazon SES email (a few thousand/month)	0.50	Rs 44
EBS weekly snapshots	1.00	Rs 88
Option B - Recommended launch total	**84.95**	**Rs 7,476**

Item	USD / month	INR / month
Everything in Option B	84.95	Rs 7,476
Application Load Balancer (estimate)	22.00	Rs 1,936
Second EC2 t3.small for high availability	16.35	Rs 1,439
Second disk + extra IPv4	5.50	Rs 484
Option C - High availability total	**128.80**	**Rs 11,334**

Recommendation: begin with Option A during development and the first admission season, then move to Option B (paid Atlas) once real student data is flowing. Option C is only needed if the site must stay up through a server failure. If you use the AWS Free Tier on a new account (t3.micro / small free hours for the first months), the first year can be cheaper.

5.2 One-time and yearly costs

Item	Estimate	Note
Domain (.com) renewal	Rs 800 - 1,200 / year	vidyanidhipucollege.com is already registered; renew with the current registrar
SSL certificates	Rs 0	AWS ACM and Let's Encrypt

Item	Estimate	Note
Google Workspace mailbox (optional)	about Rs 125 - 200 / user / month	Only if a branded college mailbox is wanted instead of Gmail
Stock photos / design assets (optional)	as needed	Campus photos from the college are preferred

5.3 Development effort (if outsourced; if you build it yourself this is your time)

Phase	Days	Cost at Rs 2,500/day
Requirements and planning	2	Rs 5,000
Environment setup	1	Rs 2,500
Backend API, email, database	4	Rs 10,000
Frontend pages and components	8	Rs 20,000
Testing and fixes	3	Rs 7,500
AWS deployment and CI/CD	3	Rs 7,500
Handover and documentation	1	Rs 2,500
Total	**22**	**Rs 55,000**

The day rate is an assumption for a junior-to-mid freelance developer in India; agency rates are usually 2-3 times higher. The starter code delivered with this document already covers a large part of the backend and frontend days.

5.4 First-year budget summary (Option B)

Component	Amount
Hosting and services, 12 months	Rs 89,707
Domain renewal (upper estimate)	Rs 1,200
Development (one-time, 5.3)	Rs 55,000
First-year total	**Rs 57,219**

Running cost after year one is the hosting line plus the domain, plus any maintenance you contract (typically 10-15% of development cost per year).

6. Timeline

Week	Work	Milestone
1	Planning, setup, backend API, SMTP	Enquiry saved in MongoDB and emailed
2-3	Frontend pages, header, common components, form integration	Full site working locally
4	Testing, content loading, AWS setup, domain and SSL	Staging site on AWS
5	Fixes, go-live, monitoring, handover	Production launch

7. Roadmap after launch

- Admin dashboard with login: view, search, filter, export enquiries to Excel, change status.
- Auto-reply email or WhatsApp message to the parent confirming the enquiry.
- Results, Blogs and News pages driven from the database (the same route / service / repository pattern).
- Online registration and fee payment, then document upload to S3.
- Analytics (Google Analytics) and SEO improvements for admission-season traffic.

8. Go-live checklist

Check	Done
Form submission stores a record and the email reaches the college inbox	[ ]
Data visible in Atlas / Compass	[ ]
HTTPS works on the main domain, www and API	[ ]
CORS allows only the real domains	[ ]
Admin routes are protected	[ ]
All pages tested on mobile	[ ]
Billing alert, backups and uptime monitoring active	[ ]
.env files are not in Git; passwords rotated from test values	[ ]

Sources for sourced figures

EC2 t3.small ap-south-1 on-demand price: doit.com compute pricing page (https://www.doit.com/compute/spot/ap-south-1/t3.small). Confirm all prices in the AWS Pricing Calculator (calculator.aws) and the MongoDB Atlas pricing page.